Arquitetura do Sistema RAG de Estudo

Guia técnico do MVP com recuperação híbrida, segurança e avaliação

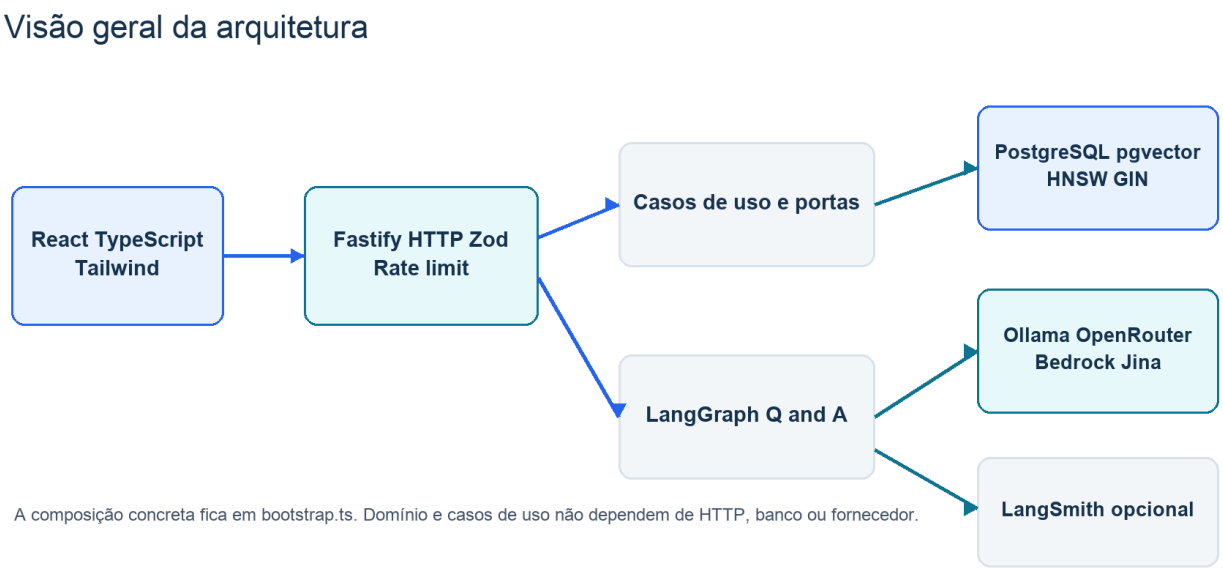


Figura 1 Componentes e dependências do sistema

Este documento descreve o sistema implementado no repositório. Ele explica o caminho completo entre a ingestão de documentos, a busca híbrida, o reranking opcional, a resposta com fontes, os controles de segurança e as avaliações que verificam regressões.

Escopo atual MVP local com Node.js, TypeScript, PostgreSQL e pgvector. Integrações externas são opcionais e exigem configuração explícita.

Objetivo e escopo

O sistema responde perguntas com base em documentos indexados. Ele não é um chatbot com memória livre: cada resposta parte de trechos recuperados, aplica uma política de segurança e retorna as fontes usadas. A arquitetura mantém o fluxo visível para estudo e permite trocar provedores sem reescrever os casos de uso.

O ambiente padrão é local e usa Ollama. OpenRouter, Amazon Bedrock, Jina e LangSmith entram por adaptadores opcionais. Nenhuma chave é enviada ao frontend.

O que acontece em uma solicitação

Etapa	Entrada	Resultado	Responsável
Ingestão	Texto e metadados	Chunks, embeddings e classificação de risco	IngestDocumentUseCase
Busca	Pergunta	Candidatos vetoriais e textuais fundidos por RRF	SearchDocumentsUseCase
Reranking	Pergunta e candidatos seguros	Ordem refinada opcional	Reranker
Q and A	Pergunta e trechos seguros	Resposta fundamentada e fontes	LangGraphRagWorkflow

Princípios do desenho

Princípio	Aplicação no MVP	Benefício
Local primeiro	Ollama e PostgreSQL executam no computador	Aprendizado reproduzível sem custo por chamada
Portas e adaptadores	Casos de uso dependem de contratos	Troca de fornecedor sem reescrever a regra
Contexto não é autoridade	Chunks são delimitados e avaliados	Reduz impacto de prompt injection

Arquitetura de camadas

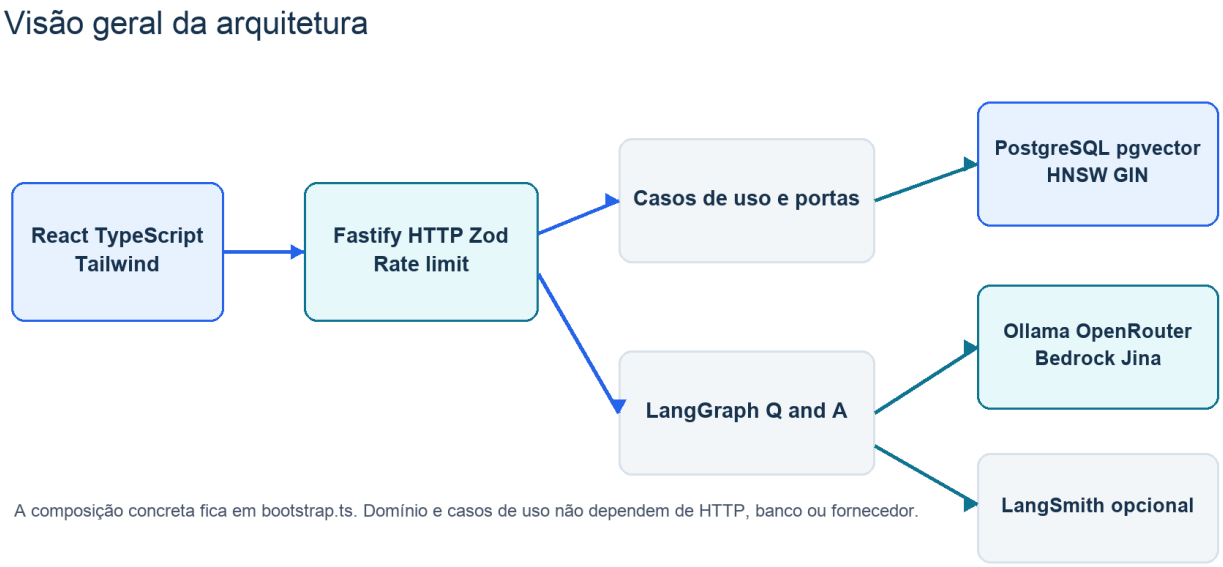


Figura 2 A aplicação depende de contratos e os adaptadores implementam integrações concretas

O domínio contém tipos e regras sem I O. A camada de aplicação coordena os casos de uso e define portas como EmbeddingProvider, DocumentRepository, AnswerGenerator e Reranker. Infraestrutura implementa essas portas com Postgres, pgvector, Ollama, OpenRouter, Bedrock e Jina. HTTP e CLI apenas traduzem entrada e saída.

bootstrap.ts é o ponto de composição. Ele escolhe o adaptador configurado por variáveis de ambiente. Esse desenho aplica inversão de dependência e evita que regras de retrieval fiquem acopladas a uma SDK específica.

Ingestão e indexação

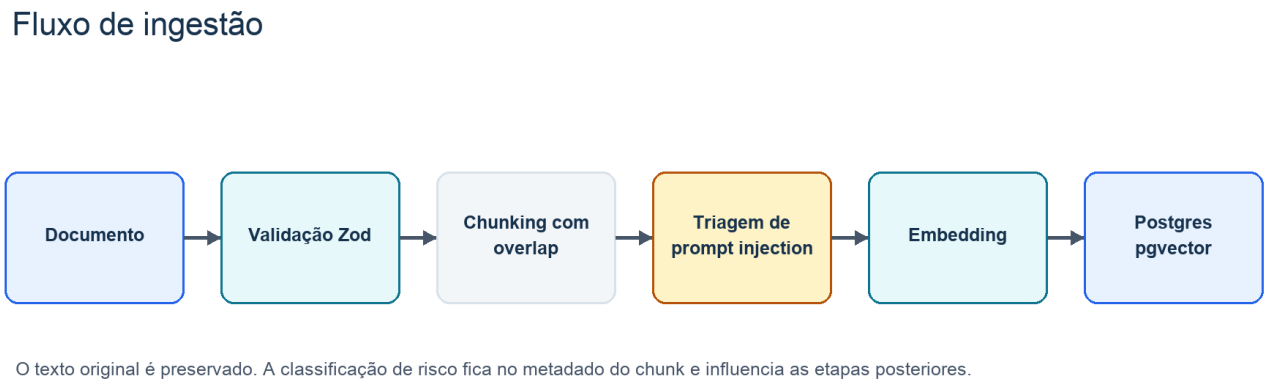


Figura 3 Caminho do documento até a persistência vetorial

O texto é validado, normalizado e dividido em chunks por sentenças. O overlap preserva ideias que cruzam a borda de um chunk. Cada chunk recebe embedding e uma avaliação de sinais de prompt injection. Os embeddings são criados antes da transação de substituição do documento, evitando apagar a versão anterior quando o provedor falha.

Escolha	Implementação	Motivo
Chunking	1.200 caracteres com 200 de overlap por padrão	Torna o mecanismo observável e evita contexto muito grande
Embeddings	1.024 dimensões	Compatível com o schema vector 1024
Substituição	Mesmo source substitui chunks em transação	Reindexação idempotente
Metadados	JSONB por documento e por chunk	Permite filtros e auditoria futura

Busca híbrida

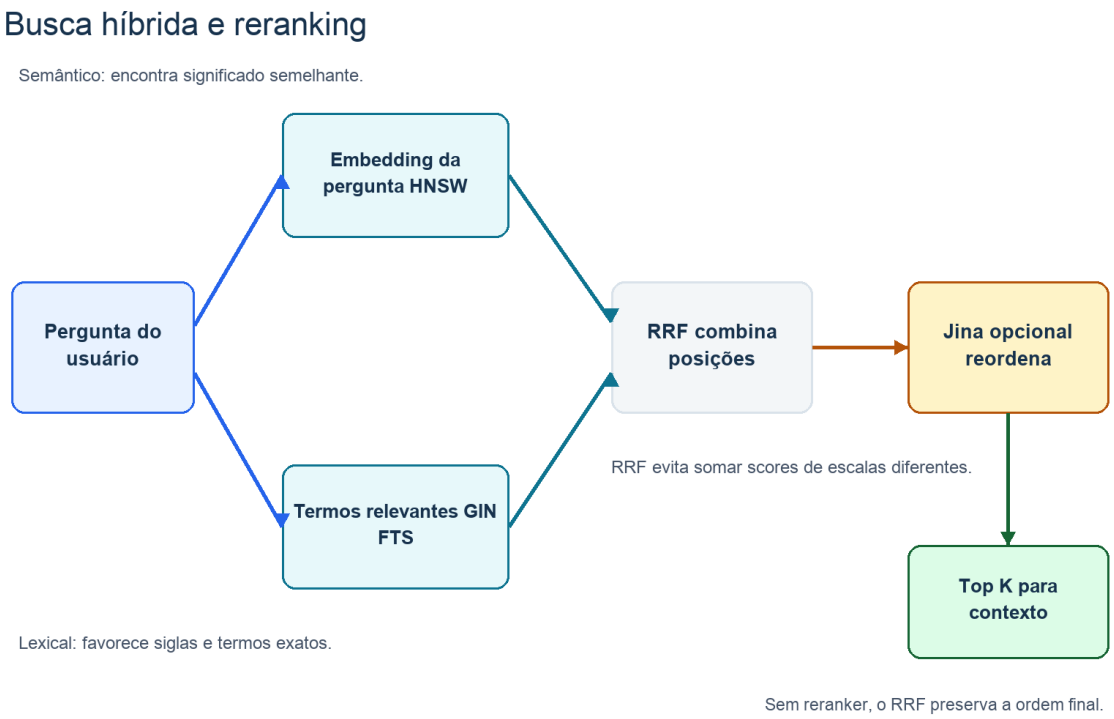


Figura 4 Dois recuperadores produzem candidatos e o RRF monta a lista final

A busca vetorial usa distância cosseno com índice HNSW. Ela encontra trechos semanticamente próximos, mesmo quando a pergunta não repete as mesmas palavras. A busca textual usa tsvector, GIN e a configuração PostgreSQL portuguese. Ela favorece siglas, nomes e frases que precisam corresponder de modo mais literal.

Os scores desses mecanismos têm escalas diferentes. Por isso o sistema não os soma. Cada mecanismo fornece uma posição no ranking e o Reciprocal Rank Fusion combina posições usando a expressão abaixo.

RRF documento = soma de 1 dividido por RRF K mais a posição no ranking

O padrão é RRF K igual a 60. Para top K igual a 5, cada recuperador fornece até 20 candidatos. A política lexical remove palavras funcionais da consulta textual, mas preserva a pergunta original para o embedding.

Reranking

RRF cria uma lista de candidatos eficiente. O reranker lê a pergunta e cada candidato em conjunto para reordenar essa lista. O projeto define a porta Reranker e inclui dois adaptadores. PassThroughReranker preserva a ordem RRF e é o padrão local. JinaReranker chama a API Jina quando RERANKER_PROVIDER recebe jina e existe uma chave no ambiente.

Modo	Dados enviados	Uso indicado	Custo
none	Nenhum dado fora da aplicação	Estudo local e baseline	Nenhum
jina	Pergunta e candidatos classificados como seguros	Comparar precisão e latência	Depende da conta Jina

O reranker nunca recebe chunks classificados como alto risco. Isso reduz a exposição a conteúdo de injection, mas não classifica dados pessoais ou confidenciais. A equipe deve decidir quais documentos podem sair da infraestrutura antes de ativar qualquer serviço externo.

Resposta com LangGraph

O fluxo de Q and A é um grafo simples e explícito. O nó retrieve chama a busca híbrida. Se nenhum chunk seguro supera o limiar semântico, o grafo encerra com uma resposta de ausência de contexto. Caso contrário, ele prepara o contexto delimitado e chama o gerador configurado.

Nó	Decisão	Saída
retrieve	Busca, reordena e remove alto risco do caminho de resposta	Matches seguros
prepare context	Respeita o limite de caracteres	Material marcado como não confiável
generate answer	Modelo responde apenas a partir do contexto	Texto e fontes do backend
no context	Não há match acima do limiar	Recusa segura

Segurança e prompt injection

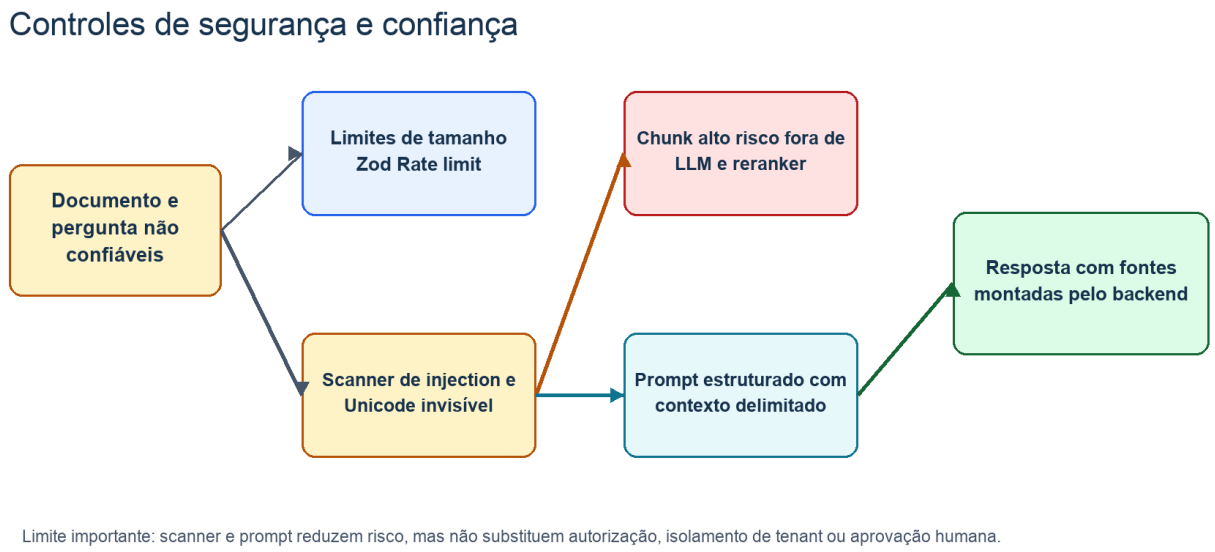


Figura 5 O documento recuperado é tratado como dado, não como autoridade

O sistema assume que documentos recuperados podem conter instruções maliciosas. Durante a ingestão, um scanner normaliza uma cópia do texto e procura sinais de override de instruções, exfiltração de segredo, alteração de papel, payload codificado e caracteres invisíveis. Um chunk de risco alto continua visível no endpoint de busca para inspeção, mas não entra no contexto do LLM nem é enviado ao reranker Jina.

O prompt de resposta fica centralizado em grounded-answer-policy.ts. Ele separa instruções confiáveis da aplicação e material recuperado não confiável. A resposta não executa ferramentas, não possui acesso a rede e não determina quais fontes serão devolvidas. O backend monta fontes a partir dos chunks selecionados.

Controle	Proteção oferecida	Limite
Zod e limites de corpo	Rejeita entradas inválidas ou grandes demais	Não entende intenção maliciosa
Scanner determinístico	Sinaliza ataques conhecidos e Unicode invisível	Pode ter falsos positivos e falsos negativos
Contexto delimitado	Reduz confusão entre dado e instrução	Não é uma fronteira de autorização
Rate limit e timeout	Reduz abuso e retenção de recursos	Não substitui autenticação
Sem ferramentas	Impede ação externa no fluxo atual	Agentes futuros exigem RBAC e aprovação

Ferramentas usadas

Camada	Ferramenta	Como é usada
Backend	Node.js e TypeScript	API tipada e execução do pipeline
HTTP	Fastify e fastify rate limit	Endpoints, body limit, headers e proteção contra abuso
Validação	Zod	Ambiente, requests, respostas de provedores e contratos de UI
Frontend	React, Vite, Tailwind e TanStack Query	Laboratório para ingerir, buscar e perguntar
Banco	PostgreSQL 16 e pgvector	Documentos, chunks, vetores, HNSW e SQL
Texto	tsvector, GIN e websearch to tsquery	Recuperação lexical em português
IA local	Ollama	Embeddings e geração no ambiente local
IA opcional	OpenRouter e Amazon Bedrock	Adapters para comparar provedores
Orquestração	LangGraph	Fluxo de busca, contexto, geração e recusa
Observabilidade	LangSmith opcional	Traces quando explicitamente ativado
Reranking	Jina opcional	Segundo estágio para ordenar candidatos seguros
Avaliação	Golden set e suíte de segurança	Regressão de retrieval e prompt injection
Infraestrutura	Docker Compose	PostgreSQL local reproduzível

Avaliação e verificação

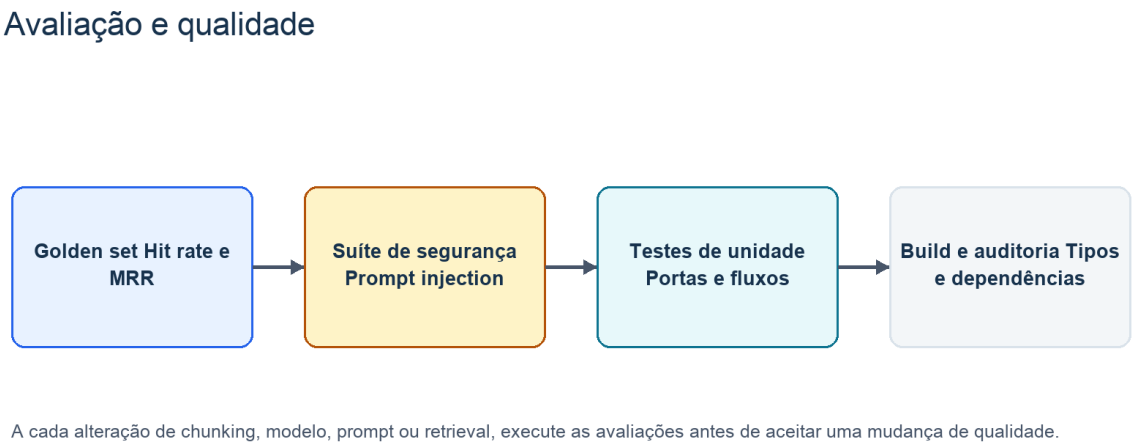


Figura 6 Avaliações atuais e pontos de expansão

O golden set compara perguntas com fontes esperadas e calcula hit rate e mean reciprocal rank. A suíte de segurança executa casos de injection e casos benignos contra o classificador. Testes de unidade exercitam casos de uso, LangGraph, política de contexto e adaptação da resposta do reranker.

Comando	Objetivo	Dependências
npm test	Testes de unidade	Nenhuma API externa
npm run eval security	Regressão do classificador de injection	Nenhuma API externa
npm run eval golden	Qualidade de retrieval real	Postgres e embedding configurado
npm run typecheck	Contratos TypeScript	Dependências Node
npm run build	Build da API e da UI	Dependências Node

Operação local

O banco Docker expõe a porta 5433 apenas em 127.0.0.1. A API usa 127.0.0.1 por padrão. O fluxo local usa Ollama e não requer AWS. A sequência de trabalho é a seguinte.

```
docker compose up -d db
npm run db:migrate
npm run ingest -- examples/rag-fundamentals.md golden/rag-fundamentals
npm run dev:api
npm run dev:web
```

Roadmap recomendado

Prioridade	Próxima evolução	Condição para adotar
1	Comparar RRF e reranker no golden set	Criar conjunto maior de perguntas reais
2	Saída estruturada com Zod e citações por chunk	Definir contrato de resposta do modelo
3	Ragas e Promptfoo	Adicionar ambiente Python e Node 22 para Promptfoo
4	Ingestão PDF DOCX e fila Redis BullMQ	Quando houver arquivos grandes e reindexação assíncrona
5	JWT, RBAC e tenant id	Antes de documentos de mais de um usuário
6	MCP e human in the loop	Depois de políticas por ferramenta e aprovação de ações

Limites atuais

O sistema é um MVP de estudo. A chave de API opcional não substitui autenticação por usuário. O rate limit é local em memória e deve migrar para uma loja compartilhada em múltiplas réplicas. O scanner de injection é uma camada de redução de risco, não uma prova de segurança. O corpus multilíngue também exigirá estratégia de idioma no índice textual.

O próximo objetivo técnico deve ser medido, não presumido: ampliar o golden set, ativar um reranker com dados permitidos e comparar hit rate, MRR, latência e custo antes de mudar o comportamento padrão.